SIMATS ENGINEERING

CSA09 – PROGRAMMING IN JAVA

ASSIGNMENT 3 – CO3 (Pair Work)

Exception Handling and Multithreading – Student A / Student B sections clearly labelled

Name	Sanjay Adithya S J
Register Number	192311336
Course Code	CSA0921
Course Outcome	CO3 – Exception handling and multithreading
GitHub Repository	https://github.com/sanoryx/CSA0921-JAVA
Pair partner (Student B)	[Partner name, register number and repository link]

AIM AND THEORY

Aim: To design, implement, test and analyse failure handling (exception hierarchy, checked/unchecked exceptions, user-defined exceptions, chaining, try-with-resources) and concurrency (threads, synchronisation, deadlock, inter-thread communication, executors) for seven realistic scenarios. **Theory:** Throwable splits into *Error* (unrecoverable, e.g. OutOfMemoryError) and *Exception*; *RuntimeException* and its subclasses are unchecked, other Exceptions are checked (must be caught or declared). catch blocks are tried in order, so narrower types must come first; *finally* always runs; try-with-resources closes AutoCloseable resources in reverse order and records close() failures as *suppressed* exceptions. A thread moves through NEW → RUNNABLE → (BLOCKED / WAITING / TIMED_WAITING) → TERMINATED. A race condition occurs when the result depends on the interleaving of unsynchronised accesses; *synchronized* gives mutual exclusion through an intrinsic lock; wait()/notifyAll() coordinate threads. All programs were executed on OpenJDK 21 on a machine with **one CPU core** (timing notes mention where this matters).

Pair-work note: both students executed every program; the contribution table at the end records the primary responsibility. Sections are labelled **[Student A]**, **[Student B]** or **[Joint]**.

Q1 – Handling Failure in a Bank ATM Simulation (6 marks)

[Joint] Failure points and classification

Failure point	Exception	Category	Caught in
Card-reader file missing	NoSuchFileException / FileNotFoundException (IOException)	checked	catch #1 (multi-catch)
Disk/read problem	IOException	checked	catch #5
Amount typed as letters	NumberFormatException	unchecked (IllegalArgumentException)	catch #3
Negative or zero amount	IllegalArgumentException	unchecked	catch #4 (multi-catch)
Unknown account	NoSuchElementException	unchecked	catch #4
Amount greater than balance	InsufficientFundsException (user-defined)	checked	catch #2
null amount	NullPointerException	unchecked	catch #6 (RuntimeException)
JVM out of memory	OutOfMemoryError	Error	not caught (cannot be recovered)

[Student A] Throwable hierarchy and catch order

Part of the Throwable hierarchy relevant to the ATM

```
Throwable
 +-- Error ....................... OutOfMemoryError, StackOverflowError
 +-- Exception
      +-- IOException (checked)
      |     +-- FileNotFoundException
      |     +-- FileSystemException -> NoSuchFileException
      +-- InsufficientFundsException (user-defined, checked)
      +-- RuntimeException (unchecked)
            +-- IllegalArgumentException -> NumberFormatException
            +-- NoSuchElementException
            +-- NullPointerException
```

Order of catch blocks: specific before general, and subclass before superclass. NumberFormatException must precede IllegalArgumentException (its parent); the generic RuntimeException is last as a safety net; IOException

(parent of the file exceptions) comes after the specific file exception. A broader type first would make the later block unreachable (compile-time error shown below). **finally guarantees** that the “card returned to customer” step executes on every path – normal completion, any caught exception and even an uncaught one – so the customer’s card is never retained.

[Student B] Test inputs, paths and stack trace

Source file: AtmDemo.java

```
import java.io.*;
import java.nio.file.*;
import java.util.*;

public class AtmDemo {
    static Map<String, Double> loadCards(Path file) throws IOException {       // checked: FileNotFoundException/IOException
        Map<String, Double> m = new HashMap<>();
        try (BufferedReader r = Files.newBufferedReader(file)) {
            for (String line; (line = r.readLine()) != null; ) { String[] p = line.split(","); m.put(p[0],
                Double.parseDouble(p[1])); }
        }
        return m;
    }
    static void withdraw(Path cardFile, String acct, String amountText) throws IOException, InsufficientFundsException {
        Map<String, Double> cards = loadCards(cardFile);
        double amount = Double.parseDouble(amountText);                        // unchecked NumberFormatException
        if (amount <= 0) throw new IllegalArgumentException("Amount must be positive");   // unchecked
        Double bal = cards.get(acct);
        if (bal == null) throw new NoSuchElementException("Unknown account " + acct);      // unchecked
        if (amount > bal) throw new InsufficientFundsException("Balance " + bal + " < " + amount);  // checked, user-defined
        System.out.println("   dispensed Rs " + amount + ", new balance " + (bal - amount));
    }
    public static void main(String[] args) throws Exception {
        Path card = Files.createTempFile("cards", ".txt");
        Files.writeString(card, "1001,5000\n1002,200\n");
        Path missing = Path.of("no-such-card-reader.txt");
        Object[][] tests = {
            {card, "1001", "1500"}, {card, "1001", "abc"}, {card, "1001", "-50"},
            {card, "1002", "900"}, {card, "9999", "100"}, {missing, "1001", "100"}, {card, "1001", null}
        };
        String[] label = {"T1 valid", "T2 letters instead of number", "T3 negative amount", "T4 more than balance",
                          "T5 unknown account", "T6 card-reader file missing", "T7 null amount"};
        for (int i = 0; i < tests.length; i++) {
            System.out.println(label[i]);
            try {
                withdraw((Path) tests[i][0], (String) tests[i][1], (String) tests[i][2]);
            } catch (FileNotFoundException | NoSuchFileException e) {                 // multi-catch (siblings, not related
                by inheritance)
                System.out.println("   catch#1 file problem: " + e.getClass().getSimpleName());
            } catch (InsufficientFundsException e) {
                System.out.println("   catch#2 InsufficientFunds: " + e.getMessage());
            } catch (NumberFormatException e) {                                       // narrower than
                IllegalArgumentException -> must come first
                System.out.println("   catch#3 NumberFormatException: " + e.getMessage());
            } catch (IllegalArgumentException | NoSuchElementException e) {
                System.out.println("   catch#4 invalid input: " + e.getMessage());
            } catch (IOException e) {
                System.out.println("   catch#5 other I/O: " + e);
            } catch (RuntimeException e) {
                System.out.println("   catch#6 unexpected runtime: " + e.getClass().getSimpleName());
            } finally {
                System.out.println("   finally: card returned to customer");
            }
        }
        Files.delete(card);
        System.out.println("\n-- uncaught exception demo (stack trace follows on stderr) --");
        level1();
    }
    static void level1() { level2(); }
    static void level2() { Integer.parseInt("12x"); }
}



class InsufficientFundsException extends Exception {
    InsufficientFundsException(String m) { super(m); }
}
```

Execution and output (7 inputs, each triggers a different path; finally executed every time):

```
$ javac AtmDemo.java && java AtmDemo
T1 valid
   dispensed Rs 1500.0, new balance 3500.0
   finally: card returned to customer
T2 letters instead of number
   catch#3 NumberFormatException: For input string: "abc"
   finally: card returned to customer
T3 negative amount
   catch#4 invalid input: Amount must be positive
   finally: card returned to customer
T4 more than balance
   catch#2 InsufficientFunds: Balance 200.0 < 900.0
```

```
   finally: card returned to customer
T5 unknown account
   catch#4 invalid input: Unknown account 9999
   finally: card returned to customer
T6 card-reader file missing
   catch#1 file problem: NoSuchFileException
   finally: card returned to customer
T7 null amount
   catch#6 unexpected runtime: NullPointerException
   finally: card returned to customer

-- uncaught exception demo (stack trace follows on stderr) --
Exception in thread "main" java.lang.NumberFormatException: For input string: "12x"
    at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
    at java.base/java.lang.Integer.parseInt(Integer.java:662)
    at java.base/java.lang.Integer.parseInt(Integer.java:778)
    at AtmDemo.level2(AtmDemo.java:57)
    at AtmDemo.level1(AtmDemo.java:56)
    at AtmDemo.main(AtmDemo.java:54)
```

Test	Input	Catch executed	finally?
T1	account 1001, 1500	none (success)	yes
T2	"abc"	#3 NumberFormatException	yes
T3	-50	#4 IllegalArgumentException	yes
T4	1002, 900 (balance 200)	#2 InsufficientFunds	yes
T5	account 9999	#4 NoSuchElementException	yes
T6	missing card file	#1 NoSuchFileException	yes
T7	null amount	#6 RuntimeException (NPE)	yes

Reading the stack trace (uncaught NumberFormatException): line 1 gives the exception type and message (*For input string:* "*12x*"); the first *at* lines are inside the JDK (forInputString, Integer.parseInt) where it was thrown; *AtmDemo.level2(AtmDemo.java:57)* is the calling line in our code; level1 and main follow – reading downward goes from the point of failure back to the start of the call chain; the uncaught exception ends the thread.

Broader catch before a narrower one – compiler message:

```
$ javac Wrong.java
Wrong.java:7: error: exception NumberFormatException has already been caught
        } catch (NumberFormatException e) {  // narrower afterwards -> unreachable
          ^
1 error
```

[Joint] Should the ATM use a single catch (Exception e)?

Only at the top-level boundary of the program (last line of defence that logs the error and shows “service unavailable”), never as a replacement for specific handlers. By catching everything the program loses the ability to react differently (retry for bad input, abort for a missing file, reject for insufficient funds), may swallow programming errors (NullPointerException) and hide the real cause. Acceptable when logging+rethrowing or at a thread/request boundary.

Alternative concept: no exceptions – hasNextInt() and error codes

The program validates before acting (`if (!sc.hasNextInt()) return -1;`) and returns codes (−1 invalid, −2 insufficient funds, −3 file missing). Every caller must remember to test the code, the number of checks grows with the number of failure kinds and code is longer; a forgotten check silently continues with bad data. Exceptions separate the normal path from the error path. **Cost:** returning a code is O(1) with no allocation; creating an exception captures the stack trace – O(d) for stack depth d and one object (with the trace array) of O(d) space. Measured in Q2 below: 1,116 ms per million exceptions with a depth-20 trace versus 0.0 ms for error codes. Exceptions are for exceptional conditions; frequent expected conditions (every other input invalid) are better checked up front.

Q2 – User-Defined Exceptions and Propagation in an Online Examination Portal (7 marks)

[Joint] Hierarchy and checked/unchecked decision

[Student A] Class diagram of the exception hierarchy

```
Exception
 +-- ExamException (checked)                    <- base of all portal business failures
       +-- WindowClosedException      (checked)   exam window closed
       +-- NotRegisteredException     (checked)   student not enrolled
       +-- FileTooLargeException      (checked)   answer file > 500 KB
       +-- SubmissionStorageException (checked)   wraps IOException from the storage layer
```

[Student A] Why checked: every rule violation is a normal, foreseeable condition the caller can recover from (show a message, ask the student to resubmit), so the compiler should force the controller to handle it.

Programming mistakes (null argument) stay unchecked. The storage exception is checked because the underlying IOException is checked and must not be lost.

[Student A] Propagation path of one exception (storage layer to controller)

```
Storage.save()  --throws-->  SubmissionStorageException(cause: IOException "Disk quota exceeded")
      |  propagates up (declared with throws)
Service.accept() --throws ExamException-->
      |
Controller.submit()  catches SubmissionStorageException -> shows "Upload failed, please retry" (cause logged)
                     catches ExamException              -> shows the rule message
```

Final handling: the controller handles it; the student sees *“Upload failed, please retry.”* The technical cause (*Disk quota exceeded*) is kept in the log only.

Source file: Q2Portal.java

```
import java.io.IOException;

public class Q2Portal {
    public static void main(String[] args) {
        Controller c = new Controller();
        String[][] scenarios = {
            {"CLOSED", "S1", "10"}, {"OPEN", "NOTREG", "10"}, {"OPEN", "S1", "900"}, {"OPEN", "QUOTA", "100"}, {"OPEN", "S1",
                 "100"}
        };
        String[] names = {"window closed", "not registered", "file too large", "disk quota full (storage failure)", "valid
            submission"};
        for (int i = 0; i < scenarios.length; i++) {
            System.out.println("Scenario " + (i + 1) + ": " + names[i]);
            c.submit(scenarios[i][0], scenarios[i][1], Long.parseLong(scenarios[i][2]));
        }
        System.out.println("\n--- chaining evidence ---");
        try { new Storage2().save("big.pdf", 900, true); }
        catch (SubmissionStorageException e) {
            System.out.println("message : " + e.getMessage());
            System.out.println("getCause: " + e.getCause());
            e.printStackTrace(System.out);
        }
        System.out.println("\n--- information lost WITHOUT the cause ---");
        try { new Storage2().save("big.pdf", 900, false); }
        catch (SubmissionStorageException e) {
            System.out.println("message : " + e.getMessage());
            System.out.println("getCause: " + e.getCause() + "   <- the disk-quota reason is gone");
            e.printStackTrace(System.out);
        }
        System.out.println("\n--- stack-trace cost per exception object (1,000,000 creations, ms, best of 5) ---");
        double with = 1e18, without = 1e18, ret = 1e18;
        for (int rep = 0; rep < 8; rep++) {
            long s = System.nanoTime(); for (int i = 0; i < 1_000_000; i++) sink += deep(20, true).hashCode(); long m =
                System.nanoTime();
            for (int i = 0; i < 1_000_000; i++) sink += deep(20, false).hashCode(); long l = System.nanoTime();
            int code = 0; for (int i = 0; i < 1_000_000; i++) code += check(i); sink += code; long e = System.nanoTime();
            if (rep >= 3) { with = Math.min(with, (m - s) / 1e6); without = Math.min(without, (l - m) / 1e6); ret =
                Math.min(ret, (e - l) / 1e6); }
        }
        System.out.printf("with stack trace (depth 20): %.0f ms   without stack trace: %.0f ms   error code: %.1f ms%n",
            with, without, ret);
    }
    static long sink;
    static Exception deep(int d, boolean trace) { return d == 0 ? (trace ? new ExamException("x") : new NoTrace("x")) :
        deep(d - 1, trace); }
    static int check(int i) { return i < 0 ? -1 : 0; }
}
class NoTrace extends RuntimeException { NoTrace(String m) { super(m, null, false, false); } }
class ExamException extends Exception {
    ExamException(String m) { super(m); }
    ExamException(String m, Throwable cause) { super(m, cause); }
}
class WindowClosedException extends ExamException { WindowClosedException() { super("Exam window has closed"); } }
class NotRegisteredException extends ExamException { NotRegisteredException(String s) { super("Student " + s + " is not
    registered for this course"); } }
class FileTooLargeException extends ExamException { FileTooLargeException(long kb) { super("Answer file of " + kb + " KB
    exceeds the 500 KB limit"); } }
class SubmissionStorageException extends ExamException {
    SubmissionStorageException(String m, Throwable cause) { super(m, cause); }
}
class Storage2 {
    void save(String name, long kb, boolean keepCause) throws SubmissionStorageException {
        try {
            if (kb > 500) throw new IOException("Disk quota exceeded");
        } catch (IOException e) {
            if (keepCause) throw new SubmissionStorageException("Could not store " + name, e);
            throw new SubmissionStorageException("Could not store " + name, null);
        }
    }
}
class Service {
    Storage2 st = new Storage2();
    void accept(String window, String student, long kb) throws ExamException {
        if (window.equals("CLOSED")) throw new WindowClosedException();
        if (student.equals("NOTREG")) throw new NotRegisteredException(student);
        if (kb > 500) throw new FileTooLargeException(kb);
        st.save("answer.pdf", student.equals("QUOTA") ? 999 : kb, true);   // "QUOTA" simulates a full disk
    }
}
class Controller {
    Service svc = new Service();
    void submit(String window, String student, long kb) {
        try { svc.accept(window, student, kb); System.out.println("   Accepted"); }
        catch (SubmissionStorageException e) { System.out.println("   Student sees: Upload failed, please retry. [internal
            cause: " + e.getCause().getMessage() + "]"); }
        catch (ExamException e) { System.out.println("   Student sees: " + e.getMessage()); }
    }
}
```

[Student B] Execution and output – one scenario per exception, chaining and the cause lost without chaining:

```
$ javac Q2Portal.java && java Q2Portal
Scenario 1: window closed
   Student sees: Exam window has closed
Scenario 2: not registered
   Student sees: Student NOTREG is not registered for this course
Scenario 3: file too large
   Student sees: Upload failed, please retry. [internal cause: Disk quota exceeded]
Scenario 4: valid submission
   Accepted

--- chaining evidence ---
message : Could not store big.pdf
getCause: java.io.IOException: Disk quota exceeded
SubmissionStorageException: Could not store big.pdf
    at Storage2.save(Q2Portal.java:59)
    at Q2Portal.main(Q2Portal.java:15)
    at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(DirectMethodHandleAccessor.java:103)
    at java.base/java.lang.reflect.Method.invoke(Method.java:580)
    at jdk.compiler/com.sun.tools.javac.launcher.Main.execute(Main.java:484)
    at jdk.compiler/com.sun.tools.javac.launcher.Main.run(Main.java:208)
    at jdk.compiler/com.sun.tools.javac.launcher.Main.main(Main.java:135)
Caused by: java.io.IOException: Disk quota exceeded
    at Storage2.save(Q2Portal.java:57)
    ... 6 more
--- information lost WITHOUT the cause ---
message : Could not store big.pdf
getCause: null   <- the disk-quota reason is gone
SubmissionStorageException: Could not store big.pdf
    at Storage2.save(Q2Portal.java:60)
    at Q2Portal.main(Q2Portal.java:22)
    at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(DirectMethodHandleAccessor.java:103)
    at java.base/java.lang.reflect.Method.invoke(Method.java:580)
    at jdk.compiler/com.sun.tools.javac.launcher.Main.execute(Main.java:484)
    at jdk.compiler/com.sun.tools.javac.launcher.Main.run(Main.java:208)
    at jdk.compiler/com.sun.tools.javac.launcher.Main.main(Main.java:135)
--- stack-trace cost per exception object (1,000,000 creations, ms, best of 5) ---
with stack trace (depth 20): 1116 ms   without stack trace: 43 ms   error code: 0.0 ms
```

Scenario	Input	Exception	Student sees
1	window CLOSED	WindowClosedException	Exam window has closed
2	student NOTREG	NotRegisteredException	Student NOTREG is not registered for this course
3	900 KB file	FileTooLargeException	Answer file of 900 KB exceeds the 500 KB limit
4	disk full	SubmissionStorageException (cause IOException)	Upload failed, please retry
5	valid	none	Accepted

[Student B] **Chaining evidence:** the first stack trace ends with *Caused by: java.io.IOException: Disk quota exceeded* – getCause() returns it. When the exception is re-thrown **without** the cause, getCause() is *null* and the stack trace shows only SubmissionStorageException: the reason (disk quota) and the original throw location are lost, so support staff cannot diagnose the failure.

Complete the Code – StorageDemo (TODO 1)

Source file: StorageDemo.java

```
import java.io.IOException;

public class StorageDemo {
    public static void main(String[] args) {
        Storage storage = new Storage();
        try {
            storage.save("answer01.pdf", 120);
            storage.save("answer02.pdf", 800);
        } catch (ExamException e) {
            System.out.println("Error: " + e.getMessage());
            System.out.println("Cause: " + e.getCause().getMessage());
        }
    }
}
class ExamException extends Exception {
    ExamException(String message) { super(message); }
    ExamException(String message, Throwable cause) { super(message, cause); }
}
class SubmissionStorageException extends ExamException {
    SubmissionStorageException(String message, Throwable cause) { super(message, cause); }
}
class Storage {
    void save(String fileName, long sizeKb) throws SubmissionStorageException {
        try {
            if (sizeKb > 500) throw new IOException("Disk quota exceeded");
            System.out.println("Saved " + fileName);
        } catch (IOException e) {
            throw new SubmissionStorageException("Could not store " + fileName, e);   // TODO 1 completed
        }
    }
}
```

Execution and output (matches the expected output):

```
$ javac StorageDemo.java && java StorageDemo
Saved answer01.pdf
Error: Could not store answer02.pdf
Cause: Disk quota exceeded
```

If the IOException were not passed to the constructor, getCause() would be null and `e.getCause().getMessage()` in main would throw a **NullPointerException** instead of printing "Cause: Disk quota exceeded".

[Joint] Why throw SubmissionStorageException instead of letting IOException reach the UI?

The user interface should not depend on storage technology; an IOException exposes implementation details (file system, quota), would force every layer to declare it and gives the student a technical message. A meaningful exception at the layer boundary expresses *what business operation failed*, hides the implementation (the storage could become a database) and still keeps the original IOException as the cause for diagnosis.

Alternative concept: unchecked custom exceptions (extends RuntimeException)

Signatures: the *throws* clauses disappear from save(), accept() and submit(); callers compile without try/catch, so a forgotten handler is found only at run time and may crash the thread. Less boilerplate, but the compiler no longer reminds the programmer. **Cost of capturing a stack trace:** Throwable.fillInStackTrace walks the whole stack – O(d) time and O(d) space for depth d, independent of checked/unchecked. Measured below: 1,000,000 exceptions with a depth-20 stack trace took 1,116 ms (1.1 µs each), the same exception with the trace disabled (*writableStackTrace* = *false*) 43 ms, an error code 0.0 ms. Chaining adds one reference per exception, so the memory of one exception is its own fields + its trace + (recursively) the cause's trace; a long cause chain keeps all those traces alive.

Q3 – Resource Safety, Debugging and Robust Code (7 marks)

[Joint] Three implementations of the sales report generator

Source file: Q3Report.java

```
import java.io.*;
import java.nio.file.*;
import java.util.*;
import java.util.logging.*;

public class Q3Report {
    static final Logger LOG = Logger.getLogger("report");
    static double sumFile(TrackedReader r) throws IOException {
        double total = 0; String line;
        while ((line = r.readLine()) != null) {
            try { total += Double.parseDouble(line.trim()); }
            catch (NumberFormatException e) { throw new IOException("Bad value '" + line + "' in " + r.name(), e); }
        }
        return total;
    }
    /** VERSION 1 - manual close() placed at the end of the try block: leaks when an exception occurs. */
    static double manualLeaky(List<Path> files) throws IOException {
        double total = 0;
        List<TrackedReader> readers = new ArrayList<>();
        for (Path p : files) readers.add(new TrackedReader(p));
        for (TrackedReader r : readers) { total += sumFile(r); r.close(); }       // exception here skips close() of this and
             every later reader
        return total;
    }
    /** VERSION 2 - manual try-catch-finally: correct but verbose. */
    static double manualFinally(List<Path> files) throws IOException {
        double total = 0;
        for (Path p : files) {
            TrackedReader r = null;
            try { r = new TrackedReader(p); total += sumFile(r); }
            finally { if (r != null) try { r.close(); } catch (IOException ignored) { } }
        }
        return total;
    }
    /** VERSION 3 - try-with-resources. */
    static double twr(List<Path> files) throws IOException {
        double total = 0;
        for (Path p : files) {
            try (TrackedReader r = new TrackedReader(p)) { total += sumFile(r); }
        }
        return total;
    }
    static void run(String label, Callable3 c, List<Path> files) {
        TrackedReader.open = 0; TrackedReader.everOpened = 0;
        try { System.out.printf("%-28s total=%.1f%n", label, c.call(files)); }
        catch (IOException e) { System.out.printf("%-28s FAILED (%s)%n", label, e.getMessage()); LOG.log(Level.WARNING,
            "failing file identified: {0}", e.getMessage()); }
        System.out.printf("%-28s handles opened=%d  still open after failure=%d%n", "", TrackedReader.everOpened,
            TrackedReader.open);
    }
    interface Callable3 { double call(List<Path> f) throws IOException; }

    // Suppressed exception demo
    static class Res implements AutoCloseable {
        final String n; Res(String n) { this.n = n; System.out.println("   open  " + n); }
        public void close() throws IOException { System.out.println("   close " + n); throw new IOException("close failed for
             " + n); }
    }

    public static void main(String[] args) throws Exception {
        Handler h = new StreamHandler(System.out, new SimpleFormatter() { public String format(LogRecord r) { return "   LOG
            " + r.getLevel() + ": " + formatMessage(r) + "\n"; } });
        LOG.setUseParentHandlers(false); LOG.addHandler(h); LOG.setLevel(Level.ALL);
        Path dir = Files.createTempDirectory("sales");
        List<Path> good = new ArrayList<>(), bad = new ArrayList<>();
        for (int i = 1; i <= 5; i++) { Path p = dir.resolve("store" + i + ".txt"); Files.writeString(p,
            "100.5\n200.0\n50.25\n"); good.add(p); }
        Path dirBad = Files.createTempDirectory("salesbad");
        for (int i = 1; i <= 5; i++) { Path p = dirBad.resolve("store" + i + ".txt"); Files.writeString(p, i == 2 ?
            "100.5\nNOT_A_NUMBER\n50\n" : "100.5\n200.0\n50.25\n"); bad.add(p); }   // store2 is corrupt
        System.out.println("== valid input: all three versions must give the same report ==");
        run("manual (leaky)", Q3Report::manualLeaky, good); run("manual try/finally", Q3Report::manualFinally, good);
            run("try-with-resources", Q3Report::twr, good);
        System.out.println("\n== corrupt store2.txt: failure part-way ==");
        run("manual (leaky)", Q3Report::manualLeaky, bad); h.flush();
        run("manual try/finally", Q3Report::manualFinally, bad); h.flush();
        run("try-with-resources", Q3Report::twr, bad); h.flush();
        System.out.println("\n== suppressed exceptions ==");
        try (Res a = new Res("A"); Res b = new Res("B")) { throw new IllegalStateException("body failed"); }
        catch (Exception e) {
            System.out.println("   primary   : " + e);
            for (Throwable s : e.getSuppressed()) System.out.println("   suppressed: " + s);
        }
        System.out.println("   (closing order is the reverse of opening: B then A)");
        System.out.println("\n== scalability: f files processed, 3 runs avg ms ==");
        for (int f : new int[]{100, 1000, 5000}) {
            List<Path> many = new ArrayList<>(); Path d2 = Files.createTempDirectory("many");
            for (int i = 0; i < f; i++) { Path p = d2.resolve("s" + i); Files.writeString(p, "1\n2\n3\n"); many.add(p); }
            double t1 = 0, t2 = 0; for (int r = 0; r < 3; r++) { long s = System.nanoTime(); manualFinally(many); long m =
                System.nanoTime(); twr(many); long e = System.nanoTime(); t1 += (m - s) / 1e6; t2 += (e - m) / 1e6; }
```

```
            System.out.printf("f=%-5d try/finally %.1f ms   try-with-resources %.1f ms   (both O(f); max handles open at once
                 = 1)%n", f, t1 / 3, t2 / 3);
        }
        System.out.println("\n== 'Too many open files' demonstration: leaky version on 5,000 valid files, ulimit -n applies
            ==");
        try { List<Path> many = new ArrayList<>(); Path d3 = Files.createTempDirectory("lim"); for (int i = 0; i < 5000; i++)
             { Path p = d3.resolve("s" + i); Files.writeString(p, "1\n"); many.add(p); }
              TrackedReader.open = 0; manualLeaky(many); System.out.println("   completed; handles open at peak were up to
                  5000 (limit not reached on this machine)"); }
        catch (IOException e) { System.out.println("   FAILED: " + e.getMessage() + " (opened " + TrackedReader.everOpened +
            ")"); }
    }
}



/** Tracks every reader that is opened and not yet closed, to count leaked file handles. */
class TrackedReader extends BufferedReader {
    static int open = 0, everOpened = 0;
    private final String name; private boolean closed;
    TrackedReader(Path p) throws IOException { super(new FileReader(p.toFile())); name = p.getFileName().toString(); open++;
        everOpened++; }
    @Override public void close() throws IOException { if (!closed) { closed = true; open--; } super.close(); }
    String name() { return name; }
}
```

Execution and output (valid input, corrupt input, suppressed exceptions, scaling and "Too many open files"):

```
$ bash -c 'ulimit -n 400; java Q3Report.java'
== valid input: all three versions must give the same report ==
manual (leaky)               total=1753.8
                             handles opened=5  still open after failure=0
manual try/finally           total=1753.8
                             handles opened=5  still open after failure=0
try-with-resources           total=1753.8
                             handles opened=5  still open after failure=0

== corrupt store2.txt: failure part-way ==
manual (leaky)               FAILED (Bad value 'NOT_A_NUMBER' in store2.txt)
                             handles opened=5  still open after failure=4
   LOG WARNING: failing file identified: Bad value 'NOT_A_NUMBER' in store2.txt
manual try/finally           FAILED (Bad value 'NOT_A_NUMBER' in store2.txt)
                             handles opened=2  still open after failure=0
   LOG WARNING: failing file identified: Bad value 'NOT_A_NUMBER' in store2.txt
try-with-resources           FAILED (Bad value 'NOT_A_NUMBER' in store2.txt)
                             handles opened=2  still open after failure=0
   LOG WARNING: failing file identified: Bad value 'NOT_A_NUMBER' in store2.txt

== suppressed exceptions ==
   open  A
   open  B
   close B
   close A
   primary   : java.lang.IllegalStateException: body failed
   suppressed: java.io.IOException: close failed for B
   suppressed: java.io.IOException: close failed for A
   (closing order is the reverse of opening: B then A)
== scalability: f files processed, 3 runs avg ms ==
f=100   try/finally 20.3 ms   try-with-resources 6.7 ms   (both O(f); max handles open at once = 1)
f=1000  try/finally 38.6 ms   try-with-resources 29.8 ms   (both O(f); max handles open at once = 1)
f=5000  try/finally 77.8 ms   try-with-resources 62.5 ms   (both O(f); max handles open at once = 1)

== 'Too many open files' demonstration: leaky version on 5,000 valid files, ulimit -n applies ==
   FAILED: /tmp/lim4873524150257957789/s394 (Too many open files) (opened 36996)
```

Version	Valid input	Corrupt store2 – handles opened / still open	Code size
1. manual close() at end of try (leaky)	total 1753.8	5 / **4**	short but wrong
2. try–catch–finally with manual close()	total 1753.8	2 / 0	long
3. try-with-resources	total 1753.8	2 / 0	shortest and correct

[Student A] Debugging and experiment

- **Where the manual version leaks:** in version 1 all five readers are opened first; the loop calls r.close() only *after* sumFile(r) returns. At store2 sumFile throws IOException (breakpoint on the throw statement, step into the caller): control leaves the method; the close() calls for store2..store5 are never executed. Setting a breakpoint on *readers.add(...)* and one in the loop and watching *TrackedReader.open* shows 5 → 4 after the failure.
- **Input that fails part-way:** corrupt line NOT_A_NUMBER in store2.txt (second of five files).
- **Files left open:** version 1 = 4; version 2 = 0; version 3 = 0 (same failing input).
- **Logging:** the catch handler logs the file that caused the failure: *LOG WARNING: failing file identified: Bad value 'NOT_A_NUMBER' in store2.txt*.
- **Real symptom reproduced:** with *ulimit -n 400* the leaky version on 5,000 files fails with *FAILED: .../s394 (Too many open files)* – exactly the production error.

[Student B] Design and correctness

- **Resources:** one reader per store file (opened) and the summary file writer; each must be closed on every path.
- **Why manual close can leak:** close() is skipped when an exception is thrown before it, and if close() itself throws, later close calls and the original exception can be lost.
- **try-with-resources guarantee:** the compiler generates a finally block that calls close() on every resource declared in the header when the block ends normally or abruptly; resources are closed in **reverse order of declaration** (shown: B closed before A).
- **Suppressed exception:** if the body throws and close() also throws, the body exception is the primary exception and the close exception is attached with addSuppressed (shown: primary IllegalStateException "body failed", suppressed IOExceptions from B and A).

[Joint] When is try-with-resources not enough?

When the resource is not AutoCloseable or is created outside the block; when many resources are opened at once (version 1 style) so the limit is exceeded before any close; when the failure leaves an *incomplete output* (a half-written summary file is still closed but is wrong – write to a temp file and rename); when the exception is swallowed; and when the process needs retry, logging and rollback logic beyond closing.

Alternative concept: explicit close() in finally, or no closing

Time: processing f files is O(f) in all versions (measured: f = 100 → 5000: try/finally 20 → 78 ms, try-with-resources 7 → 62 ms). **Maximum open handles on failure:** try-with-resources and try/finally with one file at a time = 1; the leaky version (all opened up-front, close at end) = f (5 in the test, 4 left open); **no closing at all** = every opened file stays open until garbage collection/exit = up to f handles. If f exceeds the operating-system limit (ulimit -n), the program fails with "Too many open files".

Q4 – Thread Creation and the Thread Life Cycle (7 marks)

Source file: Q4Scanners.java

```
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;

public class Q4Scanners {
    static final int[] BATCH = {40, 25, 60, 35};      // scripts per station
    static final List<String> finish = Collections.synchronizedList(new ArrayList<>());
    static final AtomicInteger total = new AtomicInteger();

    static class StationThread extends Thread {                // way 1: extend Thread
        final int id; StationThread(int id) { super("Station-" + id); this.id = id; }
        @Override public void run() { scan(id); }
    }
    static class StationTask implements Runnable {             // way 2: implement Runnable
        final int id; StationTask(int id) { this.id = id; }
        @Override public void run() { scan(id); }
    }
    static void scan(int id) {
        try { Thread.sleep(BATCH[id] * 2L); } catch (InterruptedException e) { return; }     // simulated scanning time
        total.addAndGet(BATCH[id]); finish.add("S" + id);
    }

    public static void main(String[] args) throws Exception {
        System.out.println("== A. life cycle with start(), sleep(), join(), isAlive() ==");
        Thread t = new StationThread(0);
        System.out.println("after new      : state=" + t.getState() + " alive=" + t.isAlive());
        t.start();
        System.out.println("after start()  : state=" + t.getState() + " alive=" + t.isAlive());
        Thread.sleep(20);
        System.out.println("while sleeping : state=" + t.getState() + " alive=" + t.isAlive());
        t.join();
        System.out.println("after join()   : state=" + t.getState() + " alive=" + t.isAlive());
        finish.clear(); total.set(0);

        System.out.println("\n== B. run() instead of start(): no new thread is created ==");
        Thread r = new Thread(() -> System.out.println("   body executed by thread: " + Thread.currentThread().getName()),
            "Worker-X");
        r.run();   System.out.println("   after run()  : state=" + r.getState() + "  (never started)");
        Thread r2 = new Thread(() -> System.out.println("   body executed by thread: " + Thread.currentThread().getName()),
            "Worker-X");
        r2.start(); r2.join();
        System.out.println("\n== C. four stations, join() before the total is printed ==");
        Thread[] ts = new Thread[4];
        for (int i = 0; i < 4; i++) ts[i] = (i % 2 == 0) ? new StationThread(i) : new Thread(new StationTask(i), "Station-" +
             i);
        for (Thread x : ts) x.start();
        for (Thread x : ts) x.join();
        System.out.println("   finish order: " + finish + "   total scanned = " + total.get() + " (expected 160)");

        System.out.println("\n== D. ten runs, finish order of stations (batch sizes S0=40 S1=25 S2=60 S3=35, sleep = 2
            ms/script) ==");
        Map<String, Integer> freq = new TreeMap<>();
        for (int run = 1; run <= 10; run++) {
            finish.clear(); total.set(0);
            Thread[] rs = new Thread[4];
            for (int i = 0; i < 4; i++) rs[i] = new Thread(new StationTask(i));
            for (Thread x : rs) x.start(); for (Thread x : rs) x.join();
            System.out.println("   run " + run + ": " + finish + "  total=" + total.get());
            freq.merge(finish.toString(), 1, Integer::sum);
        }
        System.out.println("   distinct orders observed: " + freq);

        System.out.println("\n== E. same work with NO sleep (pure CPU), 10 runs - order is scheduler dependent ==");
        Map<String, Integer> f2 = new TreeMap<>();
        for (int run = 1; run <= 10; run++) {
            List<String> fin = Collections.synchronizedList(new ArrayList<>()); Thread[] rs = new Thread[4];
            for (int i = 0; i < 4; i++) { final int id = i; rs[i] = new Thread(() -> { long s = 0; for (int k = 0; k <
                3_000_000; k++) s += k % 7; if (s < 0) System.out.print(""); fin.add("S" + id); }); }
            for (Thread x : rs) x.start(); for (Thread x : rs) x.join();
            f2.merge(fin.toString(), 1, Integer::sum);
        }
        System.out.println("   distinct orders: " + f2);

        System.out.println("\n== F. priorities (MAX for S3, MIN for S0) on a 4-core-independent CPU task, 10 runs ==");
        Map<String, Integer> f3 = new TreeMap<>();
        for (int run = 1; run <= 10; run++) {
            List<String> fin = Collections.synchronizedList(new ArrayList<>()); Thread[] rs = new Thread[4];
            for (int i = 0; i < 4; i++) { final int id = i; rs[i] = new Thread(() -> { long s = 0; for (int k = 0; k <
                3_000_000; k++) s += k % 7; if (s < 0) System.out.print(""); fin.add("S" + id); }); }
            rs[3].setPriority(Thread.MAX_PRIORITY); rs[0].setPriority(Thread.MIN_PRIORITY);
            for (Thread x : rs) x.start(); for (Thread x : rs) x.join();
            f3.merge(fin.toString(), 1, Integer::sum);
        }
        System.out.println("   distinct orders: " + f3 + "  (priority is only a hint to the scheduler)");

        System.out.println("\n== G. executor with fixed pool of 2 threads for 4 stations ==");
        finish.clear(); total.set(0);
        ExecutorService pool = Executors.newFixedThreadPool(2);
        long a = System.nanoTime();
```

```
        for (int i = 0; i < 4; i++) pool.submit(new StationTask(i));
        pool.shutdown(); pool.awaitTermination(5, TimeUnit.SECONDS);
        System.out.printf("   finish order %s total=%d  elapsed=%.0f ms (4 threads took about %d ms: max batch 60 x 2)%n",
            finish, total.get(), (System.nanoTime() - a) / 1e6, 120);
        System.out.println("\n== H. daemon thread: main() ends first ==");
        Thread d = new Thread(() -> { try { Thread.sleep(300); } catch (InterruptedException e) { } System.out.println("
            daemon finished its work (should NOT appear)"); });
        d.setDaemon(true); d.start();
        System.out.println("   main() ends now; daemon isDaemon=" + d.isDaemon());
    }
}
```

[Joint] Execution and output:

```
$ javac Q4Scanners.java && java Q4Scanners
== A. life cycle with start(), sleep(), join(), isAlive() ==
after new      : state=NEW alive=false
after start()  : state=RUNNABLE alive=true
while sleeping : state=TIMED_WAITING alive=true
after join()   : state=TERMINATED alive=false
== B. run() instead of start(): no new thread is created ==
   body executed by thread: main
   after run()  : state=NEW  (never started)
   body executed by thread: Worker-X
== C. four stations, join() before the total is printed ==
   finish order: [S1, S3, S0, S2]   total scanned = 160 (expected 160)

== D. ten runs, finish order of stations (batch sizes S0=40 S1=25 S2=60 S3=35, sleep = 2 ms/script) ==
   run 1: [S1, S3, S0, S2]  total=160
   run 2: [S1, S3, S0, S2]  total=160
   run 3: [S1, S3, S0, S2]  total=160
   run 4: [S1, S3, S0, S2]  total=160
   run 5: [S1, S3, S0, S2]  total=160
   run 6: [S1, S3, S0, S2]  total=160
   run 7: [S1, S3, S0, S2]  total=160
   run 8: [S1, S3, S0, S2]  total=160
   run 9: [S1, S3, S0, S2]  total=160
   run 10: [S1, S3, S0, S2]  total=160
   distinct orders observed: {[S1, S3, S0, S2]=10}

== E. same work with NO sleep (pure CPU), 10 runs - order is scheduler dependent ==
   distinct orders: {[S0, S1, S3, S2]=1, [S0, S3, S2, S1]=1, [S1, S2, S0, S3]=1, [S1, S3, S2, S0]=2, [S2, S0, S3, S1]=1, [S2,
        S3, S0, S1]=1, [S2, S3, S1, S0]=1, [S3, S0, S2, S1]=2}

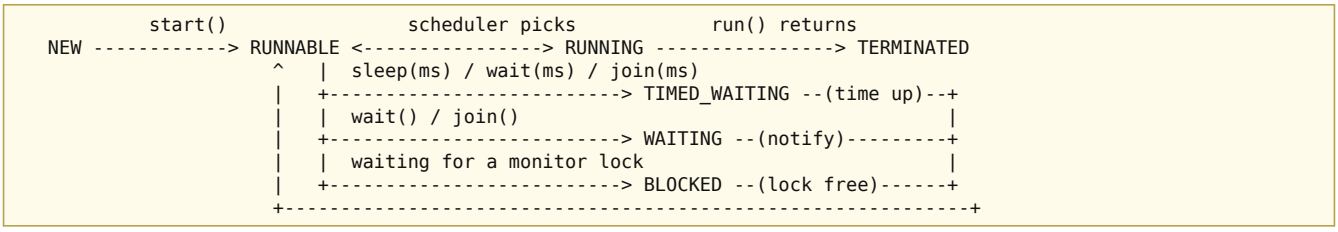
== F. priorities (MAX for S3, MIN for S0) on a 4-core-independent CPU task, 10 runs ==
   distinct orders: {[S0, S1, S2, S3]=1, [S0, S1, S3, S2]=2, [S0, S2, S1, S3]=1, [S0, S2, S3, S1]=1, [S1, S2, S0, S3]=1, [S1,
        S3, S2, S0]=1, [S2, S0, S3, S1]=1, [S2, S3, S0, S1]=1, [S3, S2, S1, S0]=1}  (priority is only a hint to the
       scheduler)
== G. executor with fixed pool of 2 threads for 4 stations ==
   finish order [S1, S0, S3, S2] total=160  elapsed=173 ms (4 threads took about 120 ms: max batch 60 x 2)

== H. daemon thread: main() ends first ==
   main() ends now; daemon isDaemon=true
```

[Student A] Thread design

- **extends Thread vs implements Runnable:** extending Thread uses up the single superclass and mixes task and thread; implementing Runnable separates the *task* from the *thread*, lets the class extend another class and lets the same task run in an Executor. Prefer Runnable (or Callable).
- **run() instead of start():** run() is an ordinary method call executed by the calling thread – evidence: *body executed by thread: main* and state stays NEW; start() created Worker-X and ran the body there. No parallelism is created by run().
- **join():** main calls join() on all four stations; it returns only when the thread is TERMINATED, so the total (160) is printed after all stations have added their counts (without join the total could be printed early).
- **Life-cycle in the test:** NEW (after construction) → RUNNABLE (after start) → TIMED_WAITING (inside sleep) → TERMINATED (after run ends; isAlive false).

[Student A] Thread life-cycle state diagram (states passed through by each station: NEW, RUNNABLE, TIMED_WAITING, RUNNABLE, TERMINATED)



[Student B] Behavioural experiments

- **Ten runs with sleep:** the finishing order was [S1, S3, S0, S2] in all 10 runs – because the sleep times (50, 70, 80, 120 ms) differ by far more than the scheduler's jitter, the order follows the work time.
- **Ten runs with pure CPU work (no sleep): 8 different orders in 10 runs** – the order depends on how the scheduler slices the threads.

- **Priorities (S3 = MAX, S0 = MIN):** 9 different orders in 10 runs and S0 (lowest priority) finished first in 5 of them – priority is only a hint, it does not control the order (and on this one-core sandbox the OS time-slices all threads anyway).
- **Daemon thread:** a daemon thread does not keep the JVM alive; main() ended and the message “daemon finished its work” was never printed – the daemon was killed when the last non-daemon thread finished.

[Joint] Why can the same program print different orders, and why must a correct program not depend on it?

The thread scheduler of the JVM/OS decides which runnable thread gets the CPU and for how long; time slices, other processes, interrupts and the number of cores vary from run to run, so the interleaving is non-deterministic. A correct program must obtain its ordering through explicit synchronisation (join, locks, latches, queues), not from timing, because otherwise it works on one machine and fails on another.

Alternative concept: ExecutorService with a fixed pool

`ExecutorService pool = Executors.newFixedThreadPool(2); pool.submit(task) … pool.shutdown(); awaitTermination()`. With n script-batches and p threads the expected completion time is about **n/p batches of work** (4 batches on 2 threads: measured 173 ms against about 120 ms with 4 threads because two stations had to wait for a free thread). Memory per thread = its stack (about 1 MB reserved by default); creating 4 explicit threads reserves 4 stacks, a pool of p threads reserves p stacks and *re-uses* them for all tasks, so for n ≫ p tasks the pool avoids n thread creations (O(n) stack allocations in the explicit version).

Q5 – Race Conditions in a Cinema Seat-Booking System (6 marks)

Complete the Code – SeatBooking (TODO 1)

Source file: SeatBooking.java

```
public class SeatBooking {
    private final boolean[] booked;
    private int bookingsMade = 0;
    SeatBooking(int seats) { booked = new boolean[seats]; }

    // TODO 1 completed: check-then-book made atomic with the intrinsic lock of this object
    synchronized boolean book(int seat) {
        if (!booked[seat]) { booked[seat] = true; bookingsMade++; return true; }
        return false;
    }

    synchronized int bookingsMadeForTest() { return bookingsMade; }
    public static void main(String[] args) throws InterruptedException {
        SeatBooking screen = new SeatBooking(10);
        Thread[] users = new Thread[100];
        for (int i = 0; i < users.length; i++) {
            final int seat = i % 10;          // 10 users compete for every seat
            users[i] = new Thread(() -> screen.book(seat));
            users[i].start();
        }
        for (Thread t : users) t.join();
        System.out.println("Seats sold: " + screen.bookingsMade);
    }
}
```

Execution and output (matches the expected output):

```
$ javac SeatBooking.java && java SeatBooking
Seats sold: 10
```

Source file: Q5Race.java

```
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;

public class Q5Race {
    interface Booker { boolean book(int seat); int sold(); }
    static class Unsync implements Booker { boolean[] b; int made; final boolean widen; Unsync(int n, boolean w) { b = new
        boolean[n]; widen = w; }
        public boolean book(int s) { if (!b[s]) { if (widen) Thread.yield(); b[s] = true; made++; return true; } return
            false; } public int sold() { return made; } }
    static class SyncMethod implements Booker { boolean[] b; int made; SyncMethod(int n) { b = new boolean[n]; }
        public synchronized boolean book(int s) { if (!b[s]) { b[s] = true; made++; return true; } return false; } public
            synchronized int sold() { return made; } }
    static class SyncBlock implements Booker { boolean[] b; int made; final Object lock = new Object(); SyncBlock(int n) { b
        = new boolean[n]; }
        public boolean book(int s) { int x = s * 31 + 7; /* non-critical work outside the lock */ synchronized (lock) { if
            (!b[s]) { b[s] = true; made++; return true; } } return false; } public int sold() { synchronized (lock) { return
            made; } } }
    static class Atomic implements Booker { AtomicBoolean[] b; AtomicInteger made = new AtomicInteger(); Atomic(int n) { b =
        new AtomicBoolean[n]; for (int i = 0; i < n; i++) b[i] = new AtomicBoolean(); }
        public boolean book(int s) { if (b[s].compareAndSet(false, true)) { made.incrementAndGet(); return true; } return
            false; } public int sold() { return made.get(); } }
    static class Reentrant implements Booker { boolean[] b; int made; final ReentrantLock l = new ReentrantLock();
        Reentrant(int n) { b = new boolean[n]; }
        public boolean book(int s) { l.lock(); try { if (!b[s]) { b[s] = true; made++; return true; } return false; } finally
             { l.unlock(); } } public int sold() { l.lock(); try { return made; } finally { l.unlock(); } } }
    static class Chm implements Booker { ConcurrentHashMap<Integer, Boolean> m = new ConcurrentHashMap<>();
        public boolean book(int s) { return m.putIfAbsent(s, true) == null; } public int sold() { return m.size(); } }
    static int[] run(Booker bk, int seats, int users) throws Exception {
        AtomicInteger successes = new AtomicInteger();
        Thread[] ts = new Thread[users];
        for (int i = 0; i < users; i++) { final int seat = i % seats; ts[i] = new Thread(() -> { if (bk.book(seat))
            successes.incrementAndGet(); }); }
        for (Thread t : ts) t.start(); for (Thread t : ts) t.join();
        return new int[]{successes.get(), bk.sold()};
    }

    public static void main(String[] args) throws Exception {
        System.out.println("== A. original test (100 threads, 10 seats) WITHOUT synchronized, 10 runs ==");
        for (int r = 1; r <= 10; r++) { int[] x = run(new Unsync(10, false), 10, 100); System.out.println("   run " + r + ":
            'successful' bookings=" + x[0] + "  bookingsMade counter=" + x[1] + (x[0] > 10 ? "   <-- DOUBLE BOOKING" : ""));
            }
        System.out.println("\n== B. same test, but a Thread.yield() inserted between the check and the update (widens the
            race window), 10 runs ==");
        int bad = 0; for (int r = 1; r <= 10; r++) { int[] x = run(new Unsync(10, true), 10, 100); if (x[0] > 10) bad++;
            System.out.println("   run " + r + ": sold=" + x[0] + "  counter=" + x[1] + (x[0] > 10 ? "   <-- DOUBLE BOOKING
            (" + (x[0] - 10) + " extra)" : "")); }
        System.out.println("   runs with double booking: " + bad + " / 10");
        System.out.println("\n== C. synchronized version (assignment program), 10 runs ==");
        for (int r = 1; r <= 10; r++) { SeatBooking s = new SeatBooking(10); Thread[] u = new Thread[100]; for (int i = 0; i
            < 100; i++) { final int seat = i % 10; u[i] = new Thread(() -> s.book(seat)); u[i].start(); } for (Thread t : u)
            t.join(); System.out.print((r == 1 ? "   Seats sold: " : ", ") + s.bookingsMadeForTest()); }
        System.out.println("   (expected 10 each time)");

        System.out.println("\n== D. 10,000 bookings: 8 threads x 1,250 attempts on 1,000 seats, time ms (best of 7); correct
            = every seat sold exactly once => sold == distinct seats ==");
        Map<String, java.util.function.IntFunction<Booker>> impl = new LinkedHashMap<>();
        impl.put("synchronized method", SyncMethod::new); impl.put("synchronized block", SyncBlock::new);
            impl.put("ReentrantLock", Reentrant::new);
        impl.put("AtomicBoolean[] CAS", Atomic::new); impl.put("ConcurrentHashMap", n -> new Chm()); impl.put("unsynchronized
             (WRONG)", n -> new Unsync(n, false));
        for (var e : impl.entrySet()) {
            double best = 1e18; int lastSold = -1;
            for (int rep = 0; rep < 7; rep++) {
                Booker b = e.getValue().apply(1000); Thread[] ts = new Thread[8]; Random seed = new Random(rep);
                long s = System.nanoTime();
                for (int i = 0; i < 8; i++) { final int off = i * 1250; ts[i] = new Thread(() -> { Random r = new
                    Random(off); for (int k = 0; k < 1250; k++) b.book(r.nextInt(1000)); }); }
                for (Thread t : ts) t.start(); for (Thread t : ts) t.join();
                best = Math.min(best, (System.nanoTime() - s) / 1e6); lastSold = b.sold();
            }
            System.out.printf("   %-24s %.2f ms   last run sold=%d%n", e.getKey(), best, lastSold);
        }
        System.out.println("\n== E. contention comparison: 4 threads x 250,000 book() calls on 10 seats, ms (best of 5) ==");
        for (var e : impl.entrySet()) {
            if (e.getKey().startsWith("unsync")) continue;
            double best = 1e18;
            for (int rep = 0; rep < 5; rep++) {
                Booker b = e.getValue().apply(10); Thread[] ts = new Thread[4]; long s = System.nanoTime();
                for (int i = 0; i < 4; i++) { ts[i] = new Thread(() -> { for (int k = 0; k < 250_000; k++) b.book(k % 10);
                    }); }
                for (Thread t : ts) t.start(); for (Thread t : ts) t.join();
                best = Math.min(best, (System.nanoTime() - s) / 1e6);
            }
            System.out.printf("   %-24s %.1f ms%n", e.getKey(), best);
        }
    }
}
```

Execution and output of the experiments:

```
$ javac SeatBooking.java Q5Race.java && java Q5Race
== A. original test (100 threads, 10 seats) WITHOUT synchronized, 10 runs ==
   run 1: 'successful' bookings=10  bookingsMade counter=10
   run 2: 'successful' bookings=10  bookingsMade counter=10
   run 3: 'successful' bookings=10  bookingsMade counter=10
   run 4: 'successful' bookings=10  bookingsMade counter=10
   run 5: 'successful' bookings=10  bookingsMade counter=10
   run 6: 'successful' bookings=10  bookingsMade counter=10
   run 7: 'successful' bookings=10  bookingsMade counter=10
   run 8: 'successful' bookings=10  bookingsMade counter=10
   run 9: 'successful' bookings=10  bookingsMade counter=10
   run 10: 'successful' bookings=10  bookingsMade counter=10
== B. same test, but a Thread.yield() inserted between the check and the update (widens the race window), 10 runs ==
   run 1: sold=20  counter=20   <-- DOUBLE BOOKING (10 extra)
   run 2: sold=24  counter=24   <-- DOUBLE BOOKING (14 extra)
   run 3: sold=34  counter=34   <-- DOUBLE BOOKING (24 extra)
   run 4: sold=10  counter=10
   run 5: sold=23  counter=23   <-- DOUBLE BOOKING (13 extra)
   run 6: sold=22  counter=22   <-- DOUBLE BOOKING (12 extra)
   run 7: sold=28  counter=28   <-- DOUBLE BOOKING (18 extra)
   run 8: sold=35  counter=35   <-- DOUBLE BOOKING (25 extra)
   run 9: sold=35  counter=35   <-- DOUBLE BOOKING (25 extra)
   run 10: sold=34  counter=34   <-- DOUBLE BOOKING (24 extra)
   runs with double booking: 9 / 10

== C. synchronized version (assignment program), 10 runs ==
   Seats sold: 10, 10, 10, 10, 10, 10, 10, 10, 10, 10   (expected 10 each time)

== D. 10,000 bookings: 8 threads x 1,250 attempts on 1,000 seats, time ms (best of 7); correct = every seat sold exactly once
     => sold == distinct seats ==
   synchronized method      0.81 ms   last run sold=1000
   synchronized block       0.65 ms   last run sold=1000
   ReentrantLock            0.99 ms   last run sold=1000
   AtomicBoolean[] CAS      0.62 ms   last run sold=1000
   ConcurrentHashMap        1.54 ms   last run sold=1000
   unsynchronized (WRONG)   0.36 ms   last run sold=1000

== E. contention comparison: 4 threads x 250,000 book() calls on 10 seats, ms (best of 5) ==
   synchronized method      12.7 ms
   synchronized block       13.2 ms
   ReentrantLock            19.2 ms
   AtomicBoolean[] CAS      7.9 ms
   ConcurrentHashMap        9.2 ms
```

[Student A] Producing and explaining the race

- **Original test without synchronized, 10 runs:** all ten runs printed 10 – the window between the check (*if (!booked[seat])*) and the write (*booked[seat] = true*) is only a few nanoseconds, so on this machine the race is rare; absence of a failure proves nothing.
- **Reliable test:** insert *Thread.yield()* between the check and the update (widening the window; the logic is unchanged). Result: **9 of 10 runs double-booked** (sold 20, 24, 34, 23, 22, 28, 35, 35, 34 instead of 10).
- **Interleaving of two threads (same seat 4):** T1 reads booked[4] = false → (switch) → T2 reads booked[4] = false → T2 writes true, bookingsMade++ → (switch) → T1 writes true, bookingsMade++ → the seat is sold twice.
- **Critical section:** the check-then-act sequence *if (!booked[seat]) { booked[seat] = true; bookingsMade++; }* (also bookingsMade++ is a read-modify-write that is itself not atomic).

[Student B] Comparing synchronisation strategies

Strategy (10,000 bookings, 8 threads, 1,000 seats)	Time (ms)	Correct?
synchronized method (locks whole method)	0.81	yes (sold = 1000)
synchronized block (only the critical section)	0.65	yes
ReentrantLock	0.99	yes
AtomicBoolean[] with compareAndSet	0.62	yes
ConcurrentHashMap.putIfAbsent	1.54	yes
unsynchronized (wrong)	0.36	sold count unreliable

Under heavy contention (4 threads × 250,000 calls on 10 seats): synchronized method 12.7 ms, synchronized block 13.2 ms, ReentrantLock 19.2 ms, AtomicBoolean CAS 7.9 ms, ConcurrentHashMap 9.2 ms. **Method vs block:** a synchronized method holds the lock for the whole method; a block holds it only around the critical section so non-critical work (here a small computation) runs in parallel; for this tiny method the difference is within noise (0.81 vs 0.65 ms). **Static synchronization** locks the Class object (*SeatBooking.class*); it is needed only if the shared data is static (e.g. one global counter for all screens) – here the data belongs to one screen (instance), so the instance lock is correct and static locking would needlessly serialise all 20 screens. **Intrinsic lock:** the synchronized method uses *this*; the block uses the lock object named in the parentheses (*lock* in my code, or *this*).

[Joint] Deployment decision for 20 screens

One **SeatBooking object per screen with a synchronized method (or a per-seat AtomicBoolean CAS)**: each screen has its own lock, so bookings on different screens never block each other; within a screen the critical section is tiny, so contention is low. CAS gives the best throughput under contention (7.9 ms), but a synchronized method is the simplest to verify and its measured cost is negligible; I would choose the synchronized method and move to CAS only if profiling shows lock contention.

Alternative concepts: AtomicBoolean[] CAS, ReentrantLock, ConcurrentHashMap.putIfAbsent

CAS: *booked[s].compareAndSet(false, true)* is an atomic check-and-set on one seat, no lock; failing threads simply see false. Time under contention: O(1) with retries only if multiple threads hit the same seat; space O(s) extra (one AtomicBoolean object, about 16 B, per seat). **ReentrantLock:** same logic as synchronized with explicit lock/unlock (try/finally), tryLock and fairness options; O(1), 1 lock object. **ConcurrentHashMap.putIfAbsent(seat, true) == null:** O(1), space O(booked seats) (only booked seats stored), finer-grained internal locking.

Q6 – Deadlock and Inter-Thread Communication (7 marks)

Complete the Code – TokenCounter (TODO 1 and TODO 2)

Source file: TokenCounter.java

```
public class TokenCounter {
    private final int[] buffer = new int[5]; // the counter holds at most 5 tokens
    private int count = 0, in = 0, out = 0;
    int maxSeen = 0, minSeen = 0;

    public synchronized void put(int token) throws InterruptedException {
        while (count == buffer.length) wait();          // TODO 1 completed: wait while full
        buffer[in] = token; in = (in + 1) % buffer.length; count++;
        maxSeen = Math.max(maxSeen, count);
        notifyAll();
    }
    public synchronized int take() throws InterruptedException {
        while (count == 0) wait();                      // TODO 2 completed: wait while empty
        int token = buffer[out]; out = (out + 1) % buffer.length; count--;
        minSeen = Math.min(minSeen, count);
        notifyAll();
        return token;
    }

    public static void main(String[] args) throws InterruptedException {
        TokenCounter counter = new TokenCounter();
        int[] collected = new int[2];
        Thread cook = new Thread(() -> {
            try {
                for (int t = 1; t <= 20; t++) counter.put(t);
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });
        Thread[] students = new Thread[2];
        for (int s = 0; s < 2; s++) {
            final int id = s;
            students[s] = new Thread(() -> {
                try {
                    for (int k = 0; k < 10; k++) collected[id] += counter.take();
                } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            });
        }
        cook.start(); for (Thread t : students) t.start();
        cook.join(); for (Thread t : students) t.join();
        System.out.println("Sum of tokens collected: " + (collected[0] + collected[1])
                + " (expected 210)");
        System.out.println("Max tokens ever in counter: " + counter.maxSeen + " (limit 5), min: " + counter.minSeen + "
            (limit 0)");
    }
}
```

Execution and output (matches the expected output; counter size never exceeded 5 or dropped below 0):

```
$ javac TokenCounter.java && java TokenCounter   # run 3 times
Sum of tokens collected: 210 (expected 210)
Max tokens ever in counter: 5 (limit 5), min: 0 (limit 0)
(three identical results)
```

[Student A] Producer–consumer design and the if-versus-while experiment

Result (the only change is while → if in both methods):

```
$ 30 runs of the version with 'if' instead of 'while'
run 1: Sum of tokens collected: 175 (expected 210) Max tokens ever in counter: 5 (limit 5), min: -6 (limit 0)
run 2: Sum of tokens collected: 75 (expected 210) Max tokens ever in counter: 5 (limit 5), min: -14 (limit 0)
run 4: Sum of tokens collected: 175 (expected 210) Max tokens ever in counter: 5 (limit 5), min: -6 (limit 0)
run 9: Sum of tokens collected: 60 (expected 210) Max tokens ever in counter: 5 (limit 5), min: -15 (limit 0)
run 11: Sum of tokens collected: 135 (expected 210) Max tokens ever in counter: 5 (limit 5), min: -10 (limit 0)
```

```
run 13: Sum of tokens collected: 175 (expected 210) Max tokens ever in counter: 5 (limit 5), min: -6 (limit 0)
run 14: Sum of tokens collected: 175 (expected 210) Max tokens ever in counter: 5 (limit 5), min: -6 (limit 0)
run 15: Sum of tokens collected: 135 (expected 210) Max tokens ever in counter: 5 (limit 5), min: -10 (limit 0)
run 20: Sum of tokens collected: 200 (expected 210) Max tokens ever in counter: 5 (limit 5), min: -2 (limit 0)
run 21: Sum of tokens collected: 175 (expected 210) Max tokens ever in counter: 5 (limit 5), min: -6 (limit 0)
run 23: Sum of tokens collected: 175 (expected 210) Max tokens ever in counter: 5 (limit 5), min: -6 (limit 0)
RESULT over 30 runs of the 'if' version: correct=19 wrong-sum/exception=11 hung=0
```

Difference: with *if*, 11 of 30 runs produced a wrong sum (60 to 200 instead of 210) and the counter dropped to **−15** (a consumer removed tokens from an empty counter). Reason: after wait() returns, the condition may no longer hold – with notifyAll() two consumers are woken for one token and the second one proceeds without re-checking (also spurious wake-ups are allowed by the JVM). The **while** loop re-tests the condition after every wake-up.

- **wait() inside a synchronized block:** the thread must own the monitor; wait() releases it atomically and re-acquires it on wake-up; otherwise IllegalMonitorStateException.
- **notify() vs notifyAll():** with several consumers notify() wakes one arbitrary thread; if it is a consumer when the buffer is full (or a producer when it is empty) the signal is wasted and the system can stall. My design uses notifyAll() so every waiter re-checks its own condition.
- **sleep() vs wait():** sleep(ms) pauses the thread, keeps all locks and ends by time; wait() releases the monitor, ends by notify/notifyAll (or timeout) and must be called while holding the lock; sleep is a Thread method, wait an Object method.

[Student B] Deadlock analysis and prevention

Source file: Q6Deadlock.java

```
import java.lang.management.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.locks.*;

public class Q6Deadlock {
    static class Acct { final int id; int bal; final ReentrantLock rl = new ReentrantLock(); Acct(int id, int b) { this.id =
        id; bal = b; } }

    static void unsafeTransfer(Acct from, Acct to, int amt, long pauseMs) throws InterruptedException {
        synchronized (from) {
            Thread.sleep(pauseMs);                              // lets the other thread grab its first lock
            synchronized (to) { from.bal -= amt; to.bal += amt; }
        }
    }
    static void orderedTransfer(Acct from, Acct to, int amt, long pauseMs) throws InterruptedException {
        Acct first = from.id < to.id ? from : to, second = first == from ? to : from;   // global lock order by account id
        synchronized (first) { Thread.sleep(pauseMs); synchronized (second) { from.bal -= amt; to.bal += amt; } }
    }
    static boolean tryTransfer(Acct from, Acct to, int amt, long timeoutMs) throws InterruptedException {
        if (from.rl.tryLock(timeoutMs, TimeUnit.MILLISECONDS)) {
            try { Thread.sleep(50);
                if (to.rl.tryLock(timeoutMs, TimeUnit.MILLISECONDS)) { try { from.bal -= amt; to.bal += amt; return true; }
                    finally { to.rl.unlock(); } }
            } finally { from.rl.unlock(); }
        }
        return false;                                          // caller may retry: no thread waits forever
    }

    public static void main(String[] args) throws Exception {
        System.out.println("== A. transfers in opposite directions without lock ordering ==");
        Acct a = new Acct(1, 1000), b = new Acct(2, 1000);
        Thread t1 = new Thread(() -> { try { unsafeTransfer(a, b, 100, 100); } catch (InterruptedException e) { } }, "T1
            (A->B)");
        Thread t2 = new Thread(() -> { try { unsafeTransfer(b, a, 50, 100); } catch (InterruptedException e) { } }, "T2
            (B->A)");
        t1.setDaemon(true); t2.setDaemon(true); t1.start(); t2.start();
        Thread.sleep(600);
        ThreadMXBean mx = ManagementFactory.getThreadMXBean(); long[] dl = mx.findDeadlockedThreads();
        System.out.println("   T1 state=" + t1.getState() + "  T2 state=" + t2.getState());
        System.out.println("   deadlocked threads detected by JVM: " + (dl == null ? 0 : dl.length));
        if (dl != null) for (ThreadInfo ti : mx.getThreadInfo(dl)) System.out.println("   " + ti.getThreadName() + " BLOCKED
            on " + ti.getLockName() + " held by " + ti.getLockOwnerName());
        System.out.println("   balances unchanged: A=" + a.bal + " B=" + b.bal);

        System.out.println("\n== B. same two transfers with a consistent lock order (lower account id first) ==");
        Acct c = new Acct(1, 1000), d = new Acct(2, 1000);
        Thread u1 = new Thread(() -> { try { orderedTransfer(c, d, 100, 100); } catch (InterruptedException e) { } });
        Thread u2 = new Thread(() -> { try { orderedTransfer(d, c, 50, 100); } catch (InterruptedException e) { } });
        u1.start(); u2.start(); u1.join(2000); u2.join(2000);
        System.out.println("   both finished: " + (!u1.isAlive() && !u2.isAlive()) + "   A=" + c.bal + " B=" + d.bal + "
            (total " + (c.bal + d.bal) + ", expected 2000)");

        System.out.println("\n== C. tryLock with timeout (alternative): same opposite transfers ==");
        Acct e = new Acct(1, 1000), f = new Acct(2, 1000);
        boolean[] ok = new boolean[2];
        Thread v1 = new Thread(() -> { try { ok[0] = tryTransfer(e, f, 100, 200); } catch (InterruptedException x) { } });
        Thread v2 = new Thread(() -> { try { ok[1] = tryTransfer(f, e, 50, 200); } catch (InterruptedException x) { } });
        v1.start(); v2.start(); v1.join(); v2.join();
        System.out.println("   transfer1 succeeded=" + ok[0] + " transfer2 succeeded=" + ok[1] + "  (a failed one is reported
             and can be retried; no thread hangs)  total=" + (e.bal + f.bal));

        System.out.println("\n== D. four Coffman conditions in program A ==");
        System.out.println("   mutual exclusion: synchronized lock held by one thread | hold and wait: T1 holds A while
            waiting for B");
        System.out.println("   no pre-emption: a monitor cannot be taken away | circular wait: T1 waits for T2's lock and T2
            for T1's");

        System.out.println("\n== E. producer-consumer with ArrayBlockingQueue(5) vs wait/notifyAll, 200,000 tokens, 1
            producer + 2 consumers, ms (best of 5) ==");
        double bq = 1e18, wn = 1e18;
        for (int rep = 0; rep < 5; rep++) {
            BlockingQueue<Integer> q = new ArrayBlockingQueue<>(5); long s = System.nanoTime();
            Thread p = new Thread(() -> { try { for (int i = 1; i <= 200_000; i++) q.put(i); q.put(-1); q.put(-1); } catch
                (InterruptedException x) { } });
            Thread[] cs = new Thread[2]; for (int i = 0; i < 2; i++) { cs[i] = new Thread(() -> { try { while (q.take() !=
                -1) { } } catch (InterruptedException x) { } }); cs[i].start(); }
            p.start(); p.join(); for (Thread t : cs) t.join(); bq = Math.min(bq, (System.nanoTime() - s) / 1e6);
            TokenCounter tc = new TokenCounter(); s = System.nanoTime();
            Thread p2 = new Thread(() -> { try { for (int i = 1; i <= 200_000; i++) tc.put(i); tc.put(-1); tc.put(-1); }
                catch (InterruptedException x) { } });
            Thread[] c2 = new Thread[2]; for (int i = 0; i < 2; i++) { c2[i] = new Thread(() -> { try { while (tc.take() !=
                -1) { } } catch (InterruptedException x) { } }); c2[i].start(); }
            p2.start(); p2.join(); for (Thread t : c2) t.join(); wn = Math.min(wn, (System.nanoTime() - s) / 1e6);
        }
        System.out.printf("   ArrayBlockingQueue %.0f ms   wait/notifyAll %.0f ms%n", bq, wn);
    }
}
```

Execution and output:

```
$ javac TokenCounter.java Q6Deadlock.java && java Q6Deadlock
== A. transfers in opposite directions without lock ordering ==
   T1 state=BLOCKED  T2 state=BLOCKED
   deadlocked threads detected by JVM: 2
   T1 (A->B) BLOCKED on Q6Deadlock$Acct@5f184fc6 held by T2 (B->A)
   T2 (B->A) BLOCKED on Q6Deadlock$Acct@3feba861 held by T1 (A->B)
   balances unchanged: A=1000 B=1000

== B. same two transfers with a consistent lock order (lower account id first) ==
   both finished: true   A=950 B=1050  (total 2000, expected 2000)

== C. tryLock with timeout (alternative): same opposite transfers ==
   transfer1 succeeded=false transfer2 succeeded=true  (a failed one is reported and can be retried; no thread hangs)
       total=2000

== D. four Coffman conditions in program A ==
   mutual exclusion: synchronized lock held by one thread | hold and wait: T1 holds A while waiting for B
   no pre-emption: a monitor cannot be taken away | circular wait: T1 waits for T2's lock and T2 for T1's
== E. producer-consumer with ArrayBlockingQueue(5) vs wait/notifyAll, 200,000 tokens, 1 producer + 2 consumers, ms (best of
    5) ==
   ArrayBlockingQueue 243 ms   wait/notifyAll 216 ms
```

- **Reliable deadlock:** T1 locks A then (after 100 ms) wants B; T2 locks B then wants A – both BLOCKED; the JVM's ThreadMXBean reports 2 deadlocked threads and which lock each holds.
- **Four conditions:** mutual exclusion (monitor held by one thread), hold-and-wait (T1 holds A while waiting for B), no pre-emption (a monitor cannot be taken away) and circular wait (T1 → T2 → T1).
- **Prevention by lock ordering:** always lock the account with the smaller id first. Both transfers now lock account 1 first, so a circular wait is impossible; both finished and the total balance stayed 2000.
- **Why the rule works:** if all threads acquire locks in one global order, a thread holding lock k only waits for locks with a larger number, so a cycle (which needs a wait from a larger to a smaller number) cannot form – the circular-wait condition is removed.

[Joint] Common ground

Both problems are about **waiting for a condition owned by another thread**: deadlock is an *infinite* mutual wait; producer–consumer waits are *finite* because each side eventually enables the other. Progress is ensured in deadlock prevention by a global lock order (no cycles) and in producer–consumer by the while-loop condition plus notifyAll() after every state change, so a waiting thread is always woken when its condition may be true.

Alternative concepts: ArrayBlockingQueue and tryLock with timeout

ArrayBlockingQueue<Integer>(5): put() blocks while full, take() blocks while empty – all waiting logic is inside the class (less error-prone: no forgotten while/notify). Measured for 200,000 tokens, 1 producer + 2 consumers: 243 ms vs 216 ms for my wait/notifyAll counter – the same order; space O(5). **tryLock(timeout):** each transfer tries to lock both accounts for 200 ms; if it cannot, it gives up and reports failure (transfer 1 failed, transfer 2 succeeded, total 2000), so no thread waits forever; cost is a possible retry and the need to handle failure. Time per put/take/transfer is O(1) for all designs; the **BlockingQueue** design is the least error-prone for producer–consumer and lock ordering is the cheapest deadlock-free design when the lock set is known in advance.

Q7 – Design a Concurrent, Fault-Tolerant Processor from First Principles (10 marks)

Stage 1 – Interpret the problem [Joint] (2)

- **Parallel tasks:** each of the 10,000 score files is independent (read, validate, compute). **Shared data:** only the result collection and the summary counters.
- **Failures:** missing file (NoSuchFileException), corrupt content (NumberFormatException), empty file or out-of-range score (validation), I/O error, an unexpected RuntimeException in a task, a task that never finishes. Each must produce a *recorded failure*, not stop the run.
- **Measurable requirements:** R1 all 10,000 files are accounted for (ok + failed = 10,000); R2 every injected fault is reported with its category; R3 one bad file never stops the run; R4 total time measured for 1, 2, 4, 8 threads; R5 summary correct on a hand-checked set.

Stage 2 – Design [Student A] (3)

Class diagram

```
ResultProcessor
  + runExecutor(files, threads) : Summary
  - process(Path) : Outcome          // Callable body: catches every failure, never throws
Outcome(file, ok, average, reason)   // immutable record = one row of the summary
Summary(ok, corrupt, missing, invalid, lost, meanOfMeans)
ExecutorService (fixed pool)  --submit(Callable<Outcome>)-->  Future<Outcome>
ConcurrentLinkedQueue<Outcome> results   // thread-safe shared collection
```

Pseudocode – process all score files

```
ALGORITHM ProcessAll(files, threads)
BEGIN
  pool <- fixed thread pool(threads); futures <- empty list
  FOR EACH file IN files DO
    futures.add( pool.submit( Callable: RETURN ProcessOne(file) ) )
  END FOR
  results <- empty concurrent queue; lost <- 0
  FOR EACH f IN futures DO
    TRY    results.add( f.get(timeout) )
    CATCH  ExecutionException OR TimeoutException  ->  lost <- lost + 1
  END FOR
  pool.shutdown()
  RETURN Summarise(results, lost)
END
ALGORITHM ProcessOne(file)
BEGIN
  TRY  read lines; validate each score (integer, 0..100); RETURN Outcome(ok, average)
  CATCH missing file -> RETURN Outcome(failed, "MISSING")
  CATCH bad number   -> RETURN Outcome(failed, "CORRUPT")
  CATCH invalid data -> RETURN Outcome(failed, "INVALID")
END
```

Concurrent collection: ConcurrentLinkedQueue – many threads add, only the main thread reads after all futures complete; it is lock-free and has no lost updates. (Results could also be returned through the Futures only; the queue is kept as the shared result store.) **Fault isolation:** process() catches all expected failures and returns an Outcome; unexpected RuntimeExceptions are captured by the Future and counted as *lost* in the summary, so one bad file never stops the run.

Stage 3 – Implement and measure [Student B] (3)

Source file: Q7Results.java

```
import java.io.*;
import java.nio.file.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.stream.*;
public class Q7Results {
    /** Result of one file: either a computed result or a failure reason. */
    record Outcome(String file, boolean ok, double average, String reason) {}

    static class ScoreFileException extends Exception { ScoreFileException(String m, Throwable c) { super(m, c); } }
    /** Reads, validates and computes. Every failure is converted into a ScoreFileException with a cause. */
    static Outcome process(Path p, int simulatedIoMs) {
        try {
            if (simulatedIoMs > 0) Thread.sleep(simulatedIoMs);           // simulates network/disk latency (stated in the
                report)
            List<String> lines = Files.readAllLines(p);
            if (lines.isEmpty()) throw new ScoreFileException("empty file", null);
            double sum = 0; int n = 0;
            for (String l : lines) {
                int v = Integer.parseInt(l.trim());
                if (v < 0 || v > 100) throw new ScoreFileException("score out of range: " + v, null);
                sum += v; n++;
            }
            return new Outcome(p.getFileName().toString(), true, sum / n, "");
        } catch (NoSuchFileException e) { return new Outcome(p.getFileName().toString(), false, 0, "MISSING");
        } catch (NumberFormatException e) { return new Outcome(p.getFileName().toString(), false, 0, "CORRUPT: " +
            e.getMessage());
        } catch (ScoreFileException e) { return new Outcome(p.getFileName().toString(), false, 0, "INVALID: " +
            e.getMessage());
        } catch (IOException e) { return new Outcome(p.getFileName().toString(), false, 0, "IO: " + e.getMessage());
        } catch (InterruptedException e) { Thread.currentThread().interrupt(); return new Outcome(p.getFileName().toString(),
             false, 0, "INTERRUPTED"); }
    }
    /** The designed solution: ExecutorService + Callable + Future; failures never escape as exceptions. */
    static Summary runExecutor(List<Path> files, int threads, int ioMs) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(threads);
        List<Future<Outcome>> fs = new ArrayList<>();
        for (Path p : files) fs.add(pool.submit(() -> process(p, ioMs)));
        ConcurrentLinkedQueue<Outcome> results = new ConcurrentLinkedQueue<>();
        int lost = 0;
        for (Future<Outcome> f : fs) {
            try { results.add(f.get(30, TimeUnit.SECONDS)); }
            catch (ExecutionException | TimeoutException e) { lost++; }          // belt and braces: an unexpected
                RuntimeException
        }
        pool.shutdown();
        return Summary.of(results, lost);
    }
    record Summary(int ok, int corrupt, int missing, int invalid, int lost, double meanOfMeans) {
        static Summary of(Collection<Outcome> os, int lost) {
            int ok = 0, c = 0, m = 0, i = 0; double s = 0;
            for (Outcome o : os) { if (o.ok()) { ok++; s += o.average(); } else if (o.reason().startsWith("MISSING")) m++;
                else if (o.reason().startsWith("CORRUPT")) c++; else i++; }
            return new Summary(ok, c, m, i, lost, ok == 0 ? 0 : s / ok);
        }
        int total() { return ok + corrupt + missing + invalid + lost; }
    }

    public static void main(String[] args) throws Exception {
        // ---------- hand-checked set ----------
        Path small = Files.createTempDirectory("small");
        Files.writeString(small.resolve("s1.txt"), "80\n90\n100\n");        // avg 90
        Files.writeString(small.resolve("s2.txt"), "50\n70\n");             // avg 60
        Files.writeString(small.resolve("s3.txt"), "75\nabc\n");            // corrupt
        Files.writeString(small.resolve("s4.txt"), "");                      // empty -> invalid
        Files.writeString(small.resolve("s5.txt"), "101\n");                 // out of range -> invalid
        List<Path> sm = new ArrayList<>(); for (int i = 1; i <= 6; i++) sm.add(small.resolve("s" + i + ".txt"));   // s6 is
            missing
        Summary hs = runExecutor(sm, 4, 0);
        System.out.println("== Hand-checked set of 6 files (expected: ok=2, corrupt=1, missing=1, invalid=2, mean of ok
            averages = 75.0) ==");
        System.out.println("   observed: " + hs);

        // ---------- 10,000 files ----------
        Path dir = Files.createTempDirectory("scores"); Random r = new Random(42); List<Path> files = new ArrayList<>();
        int expCorrupt = 0, expMissing = 0, expInvalid = 0;
        for (int i = 0; i < 10_000; i++) {
            Path p = dir.resolve("f" + i + ".txt"); files.add(p);
            int k = r.nextInt(100);
            if (k < 3) { expMissing++; continue; }                                          // never created
            StringBuilder sb = new StringBuilder(); int lines = 5 + r.nextInt(6);
            for (int j = 0; j < lines; j++) sb.append(r.nextInt(101)).append('\n');
            if (k < 6) { sb.append("##bad##\n"); expCorrupt++; } else if (k < 8) { sb.append("250\n"); expInvalid++; }
            Files.writeString(p, sb.toString());
        }
        System.out.println("\n== 10,000 files: injected missing=" + expMissing + " corrupt=" + expCorrupt + " out-of-range="
            + expInvalid + " ==");
        System.out.println("\n== Time vs pool size, real file I/O only (ms, best of 5; this machine has " +
            Runtime.getRuntime().availableProcessors() + " CPU core(s)) ==");
        Summary last = null;
        for (int th : new int[]{1, 2, 4, 8}) {
            double best = 1e18; for (int rep = 0; rep < 5; rep++) { long s = System.nanoTime(); last = runExecutor(files, th,
```

```
                0); best = Math.min(best, (System.nanoTime() - s) / 1e6); }
            System.out.printf("   %d thread(s): %.0f ms  -> %s%n", th, best, last);
        }
        System.out.println("   summary check: total=" + last.total() + " (must be 10000), missing matches injected: " +
            (last.missing() == expMissing) + ", corrupt: " + (last.corrupt() == expCorrupt) + ", invalid: " + (last.invalid()
             == expInvalid));
        System.out.println("\n== Same, with 2 ms SIMULATED I/O latency per file on the first 1,000 files (models slow network
             storage) ==");
        List<Path> first = files.subList(0, 1000);
        for (int th : new int[]{1, 2, 4, 8, 16, 64}) {
            long s = System.nanoTime(); runExecutor(first, th, 2); System.out.printf("   %d thread(s): %.0f ms%n", th,
                (System.nanoTime() - s) / 1e6);
        }
        System.out.println("\n== Alternatives on the 10,000 real files (ms, best of 5) ==");
        double ps = 1e18, tpf = 1e18; Summary sPs = null, sTpf = null;
        for (int rep = 0; rep < 5; rep++) {
            long s = System.nanoTime();
            List<Outcome> os = files.parallelStream().map(p -> process(p, 0)).collect(Collectors.toList());
            ps = Math.min(ps, (System.nanoTime() - s) / 1e6); sPs = Summary.of(os, 0);
        }
        for (int rep = 0; rep < 3; rep++) {
            long s = System.nanoTime(); ConcurrentLinkedQueue<Outcome> q = new ConcurrentLinkedQueue<>(); List<Thread> ts =
                new ArrayList<>();
            for (Path p : files) { Thread t = new Thread(() -> q.add(process(p, 0))); ts.add(t); t.start(); }
            for (Thread t : ts) t.join();
            tpf = Math.min(tpf, (System.nanoTime() - s) / 1e6); sTpf = Summary.of(q, 0);
        }
        System.out.printf("   parallelStream           %.0f ms  -> %s%n", ps, sPs);
        System.out.printf("   one thread per file      %.0f ms  -> %s (10,000 platform threads x ~1 MB reserved stack = ~10
            GB virtual, only touched pages are committed)%n", tpf, sTpf);
        Runtime rt = Runtime.getRuntime();
        System.out.println("   max heap " + rt.maxMemory() / (1024 * 1024) + " MB");
    }
}
```

Execution and output:

```
$ javac Q7Results.java && java Q7Results
== Hand-checked set of 6 files (expected: ok=2, corrupt=1, missing=1, invalid=2, mean of ok averages = 75.0) ==
   observed: Summary[ok=2, corrupt=1, missing=1, invalid=2, lost=0, meanOfMeans=75.0]

== 10,000 files: injected missing=296 corrupt=286 out-of-range=193 ==
== Time vs pool size, real file I/O only (ms, best of 5; this machine has 1 CPU core(s)) ==
   1 thread(s): 226 ms  -> Summary[ok=9225, corrupt=286, missing=296, invalid=193, lost=0, meanOfMeans=49.85223258915135]
   2 thread(s): 146 ms  -> Summary[ok=9225, corrupt=286, missing=296, invalid=193, lost=0, meanOfMeans=49.85223258915135]
   4 thread(s): 120 ms  -> Summary[ok=9225, corrupt=286, missing=296, invalid=193, lost=0, meanOfMeans=49.85223258915135]
   8 thread(s): 84 ms  -> Summary[ok=9225, corrupt=286, missing=296, invalid=193, lost=0, meanOfMeans=49.85223258915135]
   summary check: total=10000 (must be 10000), missing matches injected: true, corrupt: true, invalid: true
== Same, with 2 ms SIMULATED I/O latency per file on the first 1,000 files (models slow network storage) ==
   1 thread(s): 2222 ms
   2 thread(s): 1106 ms
   4 thread(s): 562 ms
   8 thread(s): 277 ms
   16 thread(s): 148 ms
   64 thread(s): 51 ms

== Alternatives on the 10,000 real files (ms, best of 5) ==
   parallelStream           96 ms  -> Summary[ok=9225, corrupt=286, missing=296, invalid=193, lost=0,
       meanOfMeans=49.85223258915135]
   one thread per file      894 ms  -> Summary[ok=9225, corrupt=286, missing=296, invalid=193, lost=0,
       meanOfMeans=49.85223258915135] (10,000 platform threads x ~1 MB reserved stack = ~10 GB virtual, only touched pages
       are committed)
   max heap 964 MB
```

- **Hand-checked set of 6 files** (s1 avg 90, s2 avg 60, s3 corrupt, s4 empty, s5 out of range, s6 missing): expected ok = 2, corrupt = 1, missing = 1, invalid = 2, mean of averages = 75.0 → observed exactly this.
- **10,000 files with injected faults** (296 missing, 286 corrupt, 193 out-of-range): the summary reported 296 / 286 / 193 and 9,225 successful, total 10,000 – all counts match.
- **Time vs pool size (real I/O):** 1 thread 226 ms, 2 threads 146 ms, 4 threads 120 ms, 8 threads 84 ms. With a simulated 2 ms I/O latency per file (1,000 files): 2,222 / 1,106 / 562 / 277 / 148 / 51 ms for 1 / 2 / 4 / 8 / 16 / 64 threads.
- **Unhandled failures recorded:** none during these runs (lost = 0). An unexpected exception type (e.g. OutOfMemoryError) is not an Exception and would not be caught – documented limitation.

Stage 4 – Defend the design [Joint] (2)

Recommended pool size: for I/O-bound work ≈ number of cores × (1 + wait time / compute time); the latency experiment shows near-linear gains up to 64 threads because threads mostly sleep, whereas for pure CPU-bound work on this *1-core* machine more than one thread cannot add computing power (the real-I/O gain from 226 to 84 ms came from overlapping file-system waits with computation and shrinks quickly beyond 8). Choose the smallest pool at which the curve flattens (here 8 for local files; 16–32 for network storage), because extra threads cost memory (a stack each) and context switches. **Complete and correct summary:** every file produces exactly one Outcome or one lost count and the main thread waits on every Future (with a timeout), so ok + corrupt + missing + invalid + lost = number of files (10,000 verified); results are collected in a thread-safe queue and summarised after all tasks finish. **Limitation:** the list of 10,000 Futures and Outcomes is held in

memory, and a task that hangs forever is detected only by the 30 s timeout (the worker thread is not killed).

Alternative concept: parallel streams, or one thread per file

Approach	Total time (10,000 files)	Threads / memory	Failure of one file
ExecutorService (8 threads)	84 ms	8 stacks (about 8 MB reserved)	caught in process(); Future captures anything else; run continues
parallelStream()	96 ms	common ForkJoinPool: cores − 1 workers (+ caller)	an uncaught exception in the lambda aborts the stream; here process() catches all, so the run continues
one thread per file	894 ms (10× slower)	10,000 threads × stack (about 10 GB of address space reserved; thread creation + scheduling overhead)	an uncaught exception ends only that thread and is silently lost unless an uncaught-exception handler records it

The executor gives control over the pool size, a result handle (Future) per task and a clean failure path; parallel streams are the shortest code but offer no control of the pool and hide failures; thread-per-file is O(n) threads of stack memory and is the slowest.

INDIVIDUAL CONTRIBUTION RECORD

Question	Student A – primary responsibility	Student B – primary responsibility	Joint
Q1	Throwable hierarchy, failure table, catch order, finally	Test inputs, paths, stack trace, wrong catch order	Design question, implementation
Q2	Hierarchy diagram, checked/unchecked, propagation path	Scenarios, chaining evidence, lost-cause demo	Implementation of throw/throws
Q3	Debugging, failing input, leak count, logging	Resources, TWR order, suppressed exceptions	Three implementations
Q4	Thread design, life cycle, run() vs start(), join()	Ten runs, priorities, daemon	Implementation, conclusion
Q5	Reproducing and explaining the race	Timing, method vs block, static sync	Implementation, decision
Q6	Producer–consumer design, if vs while	Deadlock, conditions, lock ordering	Implementation, conclusion
Q7	Stage 2 design	Stage 3 implementation and measurement	Stages 1 and 4

Note: add your partner's details on the first page and re-run the programs on your own machine so that the measurements are yours.